Closure & Verification Ambience

Complete FlutterFlow custom widget code

Before using: Pass **baseurl** as `https://uatapp.manappuram.net`, **branchId** as the current branch ID (example: 196), and **empCode** as the auditor ID expected by the API. The widget loads unique Audit IDs, loads the selected audit's rows, and submits all rows in one POST request.

```
// Automatic FlutterFlow imports
import '/backend/schema/structs/index.dart';
import '/flutter_flow/flutter_flow_theme.dart';
import '/flutter_flow/flutter_flow_util.dart';
import '/custom_code/widgets/index.dart'; // Imports other custom widgets
import 'package:flutter/material.dart';
// Begin custom widget code
// DO NOT REMOVE OR MODIFY THE CODE ABOVE!

import 'dart:convert';
import 'package:http/http.dart' as http;


class ClosureandVerificationAmbience extends StatefulWidget {
  const ClosureandVerificationAmbience({
    super.key,
    this.width,
    this.height,
    this.baseurl,
    this.branchId,
    this.empCode,
  });
  final double? width;
  final double? height;
  final String? baseurl;
  final String? branchId;
  final String? empCode;
  @override
  State&lt;ClosureandVerificationAmbience&gt; createState() =&gt;
      _ClosureandVerificationAmbienceState();
}

class _ClosureandVerificationAmbienceState
    extends State&lt;ClosureandVerificationAmbience&gt; {
  List&lt;String&gt; _auditIds = [];
  String? _selectedAuditId;
  List&lt;_AuditRow&gt; _rows = [];

  bool _isLoadingList = false;
  bool _isLoadingDetails = false;
  bool _isSubmitting = false;
  String? _errorMessage;

  // Per-row inputs (same index as _rows)
  // status: 1 = Satisfactory closed, 2 = Wrong Rectification
  List&lt;TextEditingController&gt; _remarkControllers = [];
  List&lt;int&gt; _statuses = [];

  String get _baseUrl =&gt; (widget.baseurl ?? '').replaceAll(RegExp(r'/$'), '');

  void _disposeRowControllers() {
    for (final c in _remarkControllers) {
      c.dispose();
    }
    _remarkControllers = [];
    _statuses = [];
  }

  @override
  void dispose() {
    _disposeRowControllers();
    super.dispose();
  }

  @override
  void initState() {
    super.initState();
    _loadAuditIds();
  }

  // On load: fetch Audit IDs for the branch
  Future&lt;void&gt; _loadAuditIds() async {
    final branchId = widget.branchId?.trim();
    if (_baseUrl.isEmpty || branchId == null || branchId.isEmpty) {
      setState(() =&gt; _errorMessage = 'Branch and API URL are required.');
      return;
    }
    setState(() {
      _isLoadingList = true;
```

```
    _errorMessage = null;
  });
  try {
    final response = await http.get(Uri.parse(
      '$_baseUrl/AuditInspectionApi/api/GetInspectionData/'
      'Audit_id_ambience/$branchId/1',
    ));
    if (response.statusCode &lt; 200 || response.statusCode &gt;= 300) {
      throw Exception('Unable to load audit ids.');
    }
    final result = _resultList(response.body);
    if (!mounted) return;

    // API returns duplicate AUDIT_IDs, so de-duplicate (keeps order)
    final ids = &lt;String&gt;[];
    for (final row in result) {
      if (row is Map&lt;String, dynamic&gt;) {
        final id = '${row['AUDIT_ID'] ?? ''}'.trim();
        if (id.isNotEmpty &amp;&amp; !ids.contains(id)) ids.add(id);
      }
    }
    setState(() {
      _auditIds = ids;
      _isLoadingList = false;
    });
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _isLoadingList = false;
      _errorMessage = 'Unable to load audit id list.';
    });
  }
}

// On select: fetch the datatable for the chosen Audit ID
Future&lt;void&gt; _loadDetails(String auditId) async {
  final branchId = widget.branchId?.trim();
  if (_baseUrl.isEmpty || branchId == null || branchId.isEmpty) return;
  setState(() {
    _disposeRowControllers();
    _selectedAuditId = auditId;
    _rows = [];
    _isLoadingDetails = true;
    _errorMessage = null;
  });
  try {
    final response = await http.get(Uri.parse(
      '$_baseUrl/AuditInspectionApi/api/GetInspectionData/'
      'Audit_id_Datatable/$auditId*$branchId/1',
    ));
    if (response.statusCode &lt; 200 || response.statusCode &gt;= 300) {
      throw Exception('Unable to load audit data.');
    }
    final rows = _resultList(response.body)
        .whereType&lt;Map&lt;String, dynamic&gt;&gt;()
        .map(_AuditRow.fromJson)
        .toList();
    if (!mounted) return;
    setState(() {
      _rows = rows;
      _remarkControllers =
          List.generate(rows.length, (_) =&gt; TextEditingController());
      _statuses = List.filled(rows.length, 1);
      _isLoadingDetails = false;
    });
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _isLoadingDetails = false;
      _errorMessage = 'Unable to load details for the selected audit id.';
    });
  }
}

List&lt;dynamic&gt; _resultList(String body) {
  final decoded = jsonDecode(body);
  return decoded is Map&lt;String, dynamic&gt; &amp;&amp; decoded['result'] is List
      ? decoded['result'] as List&lt;dynamic&gt;
      : &lt;dynamic&gt;[];
}

Future&lt;void&gt; _showAlert(String message) async {
  if (!mounted) return;
  await showDialog&lt;void&gt;(
    context: context,
    builder: (context) =&gt; AlertDialog(
      title: const Text('Alert'),
      content: Text(message),
      actions: [
        TextButton(
          onPressed: () =&gt; Navigator.of(context).pop(),
```

```
          child: const Text('OK'),
        ),
      ],
    ),
  );
}

Future&lt;void&gt; _submitClosure(String flag) async {
  final auditId = _selectedAuditId;
  final branchId = widget.branchId?.trim();
  final auditorId = widget.empCode?.trim();

  if (auditId == null || branchId == null || branchId.isEmpty) {
    await _showAlert('Please select an audit id first.');
    return;
  }

  if (_rows.isEmpty) {
    await _showAlert('No audit rows to verify.');
    return;
  }

  if (auditorId == null || auditorId.isEmpty) {
    await _showAlert('Auditor ID is required.');
    return;
  }

  // Every row must have its own remarks.
  for (var i = 0; i &lt; _rows.length; i++) {
    if (_remarkControllers[i].text.trim().isEmpty) {
      await _showAlert(
        'Please enter remarks for row ${i + 1} (${_rows[i].category}).',
      );
      return;
    }
  }

  setState(() {
    _isSubmitting = true;
    _errorMessage = null;
  });

  try {
    // Format: auditorid^categoryid~questionid~auditorremark~status!...
    final pageValue = StringBuffer()..write('${auditorId}^');

    for (var i = 0; i &lt; _rows.length; i++) {
      final row = _rows[i];
      final remarks = _remarkControllers[i].text.trim();
      final verificationStatus = _statuses[i];
      pageValue.write(
        '${row.categoryId}~'
        '${row.questionId}~'
        '$remarks~'
        '$verificationStatus!',
      );
    }

    // Submit all table rows in ONE POST request.
    final response = await http.post(
      Uri.parse('$_baseUrl/AuditInspectionApi/api/PostInseptionCtl'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'flag': 'Audit_id_Datatable_post',
        'pageValue': pageValue.toString(),
        'paraValue': '1',
      }),
    );

    if (response.statusCode &lt; 200 || response.statusCode &gt;= 300) {
      throw Exception('Unable to submit audit verification.');
    }

    final decoded = jsonDecode(response.body);
    var responseMessage = 'Successfully updated status.';

    if (decoded is List &amp;&amp; decoded.isNotEmpty &amp;&amp; decoded.first is Map) {
      responseMessage =
          '${decoded.first['RES'] ?? responseMessage}';
    } else if (decoded is Map) {
      responseMessage = '${decoded['RES'] ?? responseMessage}';
    }

    if (!mounted) return;

    setState(() {
      _isSubmitting = false;
      _errorMessage = null;
      _disposeRowControllers();
      _selectedAuditId = null;
      _rows = [];
    });
```

```
    await _showAlert(responseMessage);
    if (!mounted) return;
    await _loadAuditIds();
  } catch (e) {
    if (!mounted) return;
    setState(() {
      _isSubmitting = false;
      _errorMessage = 'Unable to submit the closure action.';
    });
    await _showAlert('Unable to submit audit verification.');
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF7F8FA),
    appBar: AppBar(
      leading: IconButton(
        tooltip: 'Back',
        icon: const Icon(Icons.arrow_back, color: Color(0xFF1A202C)),
        onPressed: () =&gt; Navigator.of(context).maybePop(),
      ),
      centerTitle: true,
      title: const Text(
        'Security &amp; Ambiance Closure',
        style: TextStyle(
          fontWeight: FontWeight.w600,
          fontSize: 20,
          color: Color(0xFF1A202C),
        ),
      ),
      backgroundColor: Colors.white,
      surfaceTintColor: Colors.transparent,
      shadowColor: Colors.black12,
    ),
    body: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Expanded(
          child: SingleChildScrollView(
            padding: const EdgeInsets.symmetric(
              horizontal: 48.0,
              vertical: 32.0,
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                if (_errorMessage != null)
                  Padding(
                    padding: const EdgeInsets.only(bottom: 16.0),
                    child: Text(
                      _errorMessage!,
                      style: TextStyle(
                        color: Colors.red.shade700,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ),
                _buildAuditSelectionSection(),
                if (_isLoadingDetails)
                  const Padding(
                    padding: EdgeInsets.symmetric(vertical: 40),
                    child: Center(
                      child: CircularProgressIndicator(
                        color: Color(0xFF005FB8),
                      ),
                    ),
                  ),
                if (_selectedAuditId != null &amp;&amp; !_isLoadingDetails) ...[
                  const SizedBox(height: 32),
                  _buildAuditTableSection(),
                  const SizedBox(height: 24),
                  _buildActionButtons(),
                  const SizedBox(height: 32),
                ],
              ],
            ),
          ),
        ),
      ],
    ),
  );
}

Widget _buildAuditSelectionSection() {
  return _SectionContainer(
    title: 'Audit Selection',
    icon: Icons.assignment_ind_outlined,
    child: Row(
      mainAxisAlignment: MainAxisAlignment.start,
```

```
      children: [
        _FormDropdown&lt;String&gt;(
          label: 'Select Audit ID',
          width: 260,
          value: _selectedAuditId,
          items: _auditIds,
          onChanged: _isLoadingList
              ? null
              : (value) {
                  if (value != null) _loadDetails(value);
                },
          displayText: (item) =&gt; item,
          hintText: _isLoadingList ? 'Loading...' : 'Select...',
        ),
      ],
    ),
  );
}

Widget _buildAuditTableSection() {
  const headStyle = TextStyle(
    fontSize: 12,
    fontWeight: FontWeight.w700,
    color: Color(0xFF2D3748),
  );
  const cellStyle = TextStyle(fontSize: 12, color: Color(0xFF1A202C));

  Widget cell(String text, double w) =&gt; ConstrainedBox(
        constraints: BoxConstraints(maxWidth: w),
        child: Text(text.isEmpty ? '-' : text, style: cellStyle),
      );

  return _SectionContainer(
    title: 'Audit Details',
    icon: Icons.table_chart_outlined,
    child: _rows.isEmpty
        ? const Padding(
            padding: EdgeInsets.symmetric(vertical: 16),
            child: Center(
              child: Text(
                'No data found for this audit id.',
                style: TextStyle(color: Color(0xFF4A5568)),
              ),
            ),
          )
        : LayoutBuilder(
            builder: (context, constraints) =&gt; SingleChildScrollView(
              scrollDirection: Axis.horizontal,
              child: ConstrainedBox(
                constraints: BoxConstraints(minWidth: constraints.maxWidth),
                child: DataTable(
                  headingRowColor:
                      WidgetStateProperty.all(const Color(0xFFF9FAFB)),
                  dataRowMinHeight: 40,
                  dataRowMaxHeight: 72,
                  columnSpacing: 32,
                  border: const TableBorder(
                    horizontalInside:
                        BorderSide(color: Color(0xFFEDF2F7)),
                  ),
                  columns: const [
                    DataColumn(label: Text('Sr.', style: headStyle)),
                    DataColumn(label: Text('Category', style: headStyle)),
                    DataColumn(label: Text('Question', style: headStyle)),
                    DataColumn(
                        label: Text('Audit Remark', style: headStyle)),
                    DataColumn(label: Text('BH Answer', style: headStyle)),
                    DataColumn(
                        label: Text('Auditor Remarks', style: headStyle)),
                    DataColumn(
                        label:
                            Text('Verification Status', style: headStyle)),
                  ],
                  rows: [
                    for (var i = 0; i &lt; _rows.length; i++)
                      DataRow(cells: [
                        DataCell(Text('${i + 1}', style: cellStyle)),
                        DataCell(cell(_rows[i].category, 180)),
                        DataCell(cell(_rows[i].question, 380)),
                        DataCell(cell(_rows[i].auditorRemark, 220)),
                        DataCell(cell(_rows[i].bhAnswer, 160)),
                        DataCell(_remarkField(i)),
                        DataCell(_statusDropdown(i)),
                      ]),
                  ],
                ),
              ),
            ),
          ),
  );
}
```

```
Widget _remarkField(int i) {
  return SizedBox(
    width: 260,
    height: 36,
    child: TextField(
      controller: _remarkControllers[i],
      style: const TextStyle(fontSize: 12, color: Colors.black),
      decoration: InputDecoration(
        hintText: 'Enter remarks...',
        hintStyle: const TextStyle(fontSize: 12, color: Colors.grey),
        filled: true,
        fillColor: Colors.white,
        contentPadding: const EdgeInsets.symmetric(horizontal: 10),
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(6),
          borderSide: const BorderSide(color: Color(0xFFCBD5E0)),
        ),
        enabledBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(6),
          borderSide: const BorderSide(color: Color(0xFFCBD5E0)),
        ),
        focusedBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(6),
          borderSide: const BorderSide(color: Color(0xFF005FB8), width: 1.5),
        ),
      ),
    ),
  );
}

Widget _statusDropdown(int i) {
  return Container(
    width: 200,
    height: 36,
    padding: const EdgeInsets.symmetric(horizontal: 10),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(6),
      border: Border.all(color: const Color(0xFFCBD5E0)),
    ),
    child: DropdownButtonHideUnderline(
      child: DropdownButton&lt;int&gt;(
        value: _statuses[i],
        isExpanded: true,
        icon: const Icon(Icons.arrow_drop_down, color: Color(0xFF4A5568)),
        style: const TextStyle(fontSize: 12, color: Color(0xFF1A202C)),
        items: const [
          DropdownMenuItem(value: 1, child: Text('Satisfactory closed')),
          DropdownMenuItem(value: 2, child: Text('Wrong Rectification')),
        ],
        onChanged: (value) {
          if (value != null) setState(() =&gt; _statuses[i] = value);
        },
      ),
    ),
  );
}

Widget _buildActionButtons() {
  return Row(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      ElevatedButton.icon(
        onPressed: _isSubmitting ? null : () =&gt; _submitClosure('0'),
        icon: const Icon(Icons.check),
        label: const Text('Confirm Validation'),
        style: ElevatedButton.styleFrom(
          backgroundColor: const Color(0xFF16803C),
          foregroundColor: Colors.white,
          padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(6),
          ),
        ),
      ),
      const SizedBox(width: 16),
      OutlinedButton.icon(
        onPressed: _isSubmitting ? null : () =&gt; context.go('/menuPage'),
        icon: const Icon(Icons.exit_to_app),
        label: const Text('Exit'),
        style: OutlinedButton.styleFrom(
          foregroundColor: const Color(0xFF2D3748),
          padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
          side: const BorderSide(color: Color(0xFFCBD5E0)),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(6),
          ),
        ),
      ),
    ],
  );
}
```

```
}

// --- Reusable UI Components
class _SectionContainer extends StatelessWidget {
  final String title;
  final IconData icon;
  final Widget child;
  final EdgeInsetsGeometry padding;
  const _SectionContainer({
    required this.title,
    required this.icon,
    required this.child,
    this.padding = const EdgeInsets.all(24.0),
  });
  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(12),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.05),
            blurRadius: 20,
            offset: const Offset(0, 5),
          ),
        ],
        border: Border.all(color: const Color(0xFFEDF2F7)),
      ),
      child: Column(
        children: [
          const SizedBox(height: 8),
          Container(
            padding: const EdgeInsets.symmetric(
              horizontal: 24.0,
              vertical: 16.0,
            ),
            decoration: const BoxDecoration(
              color: Color(0xFFF9FAFB),
              borderRadius: BorderRadius.only(
                topLeft: Radius.circular(12),
                topRight: Radius.circular(12),
              ),
              border: Border(
                bottom: BorderSide(color: Color(0xFFEDF2F7)),
              ),
            ),
            child: Row(
              children: [
                Icon(
                  icon,
                  size: 22,
                  color: const Color(0xFF4A5568),
                ),
                const SizedBox(width: 12),
                Text(
                  title,
                  style: const TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w600,
                    color: Color(0xFF2D3748),
                  ),
                ),
              ],
            ),
          ),
          Padding(
            padding: padding,
            child: child,
          ),
        ],
      ),
    );
  }
}

class _FormDropdown&lt;T&gt; extends StatelessWidget {
  final String label;
  final double width;
  final T? value;
  final List&lt;T&gt; items;
  final void Function(T?)? onChanged;
  final String Function(T) displayText;
  final String hintText;
  const _FormDropdown({
    Key? key,
    required this.label,
    required this.width,
    required this.value,
    required this.items,
    required this.onChanged,
```

```
    required this.displayText,
    this.hintText = 'Select...',
  }) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: width,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            label,
            style: const TextStyle(
              fontSize: 11,
              fontWeight: FontWeight.w600,
              color: Color(0xFF4A5568),
            ),
          ),
          const SizedBox(height: 8),
          Container(
            height: 36,
            padding: const EdgeInsets.symmetric(horizontal: 10),
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(6),
              border: Border.all(color: const Color(0xFFCBD5E0)),
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withOpacity(0.02),
                  blurRadius: 4,
                  offset: const Offset(0, 2),
                ),
              ],
            ),
            child: DropdownButtonHideUnderline(
              child: DropdownButton&lt;T&gt;(
                hint: Text(
                  hintText,
                  style: const TextStyle(fontSize: 12, color: Colors.grey),
                ),
                value: value,
                isExpanded: true,
                icon:
                    const Icon(Icons.arrow_drop_down, color: Color(0xFF4A5568)),
                items: items.map((item) {
                  return DropdownMenuItem&lt;T&gt;(
                    value: item,
                    child: Text(
                      displayText(item),
                      style: const TextStyle(fontSize: 12),
                      overflow: TextOverflow.ellipsis,
                    ),
                  );
                }).toList(),
                onChanged: onChanged,
                style: const TextStyle(
                  fontSize: 12,
                  color: Color(0xFF1A202C),
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

// --- Data Models
class _AuditRow {
  const _AuditRow({
    required this.categoryId,
    required this.questionId,
    required this.category,
    required this.question,
    required this.auditorRemark,
    required this.bhAnswer,
  });

  final int categoryId;
  final String questionId;
  final String category;
  final String question;
  final String auditorRemark;
  final String bhAnswer;
  factory _AuditRow.fromJson(Map&lt;String, dynamic&gt; json) =&gt; _AuditRow(
        categoryId: int.tryParse('${json['CATEGORYID'] ?? 0}') ?? 0,
        questionId: '${json['QUESTIONID'] ?? ''}',
        category: '${json['CATEGORY'] ?? ''}',
        question: '${json['QUESTION'] ?? ''}',
        auditorRemark: '${json['AUDITOR_REMARK'] ?? ''}',
```

```
        bhAnswer: '${json['BH_ANSWER'] ?? ''}',
      );
}
```